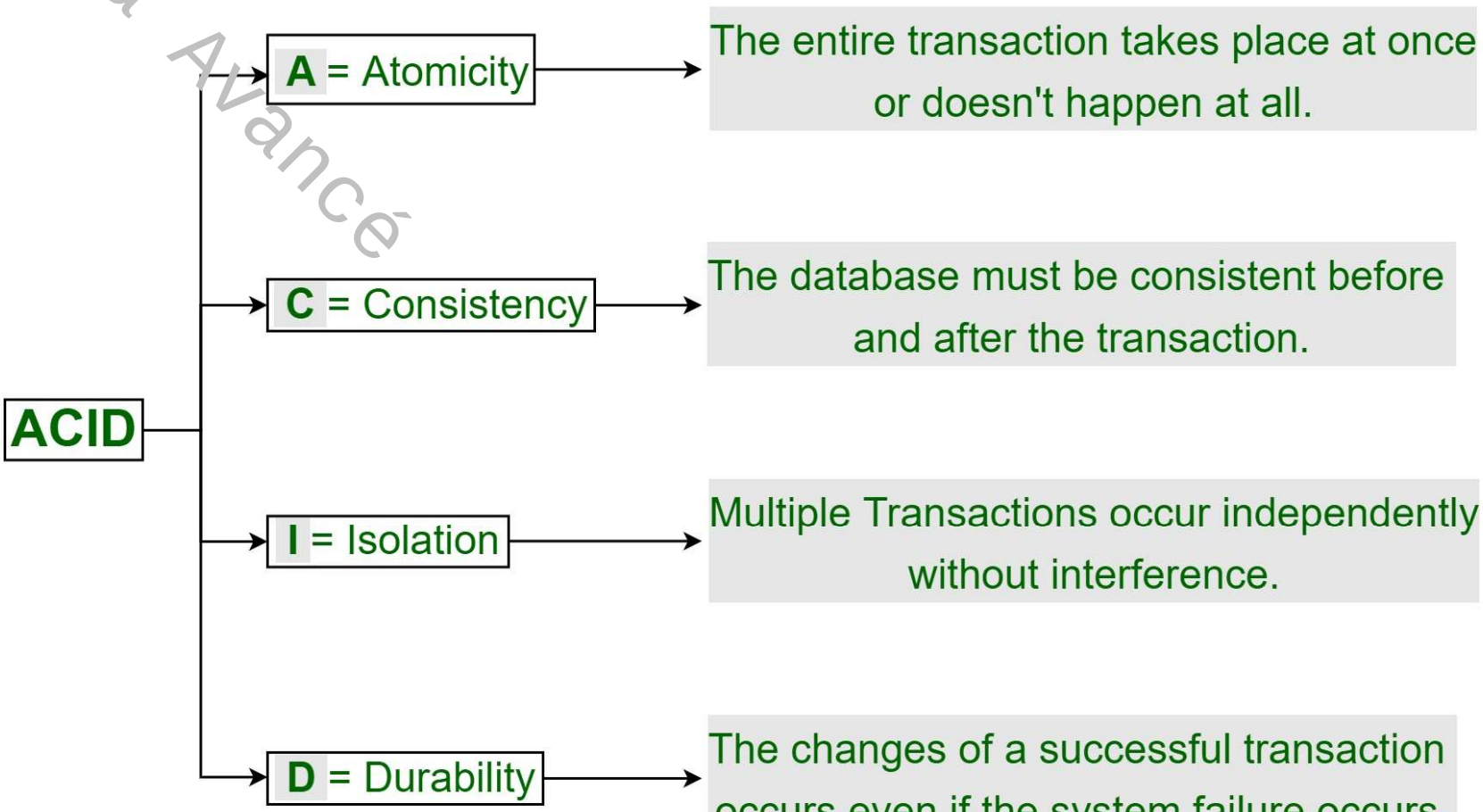
CH
HBase

- Theorem: ACID & CAP
- Limitations of HDFS
- Hbase: presentation
- Hbase : data model
- Hbase : Architecture
- Hbase : interaction & shell commands
- Annex : ACID Theorem
- Annex : CAP Theorem

ACID theorem

- The **ACID** theorem is a set of properties that guarantee the reliability of transactions in relational databases. These properties are :
  - Atomicity: A transaction is executed **entirely or not at all**. If part of the transaction fails, the entire transaction fails and the database remains unchanged.
  - Consistency: The database is always in a consistent state after a transaction has been executed.
  - Isolation: Transactions are executed in isolation from each other. Changes made by a transaction are only visible to that transaction until it is committed.
  - Durability: Changes made by a transaction are persistent, even in the event of a system failure.

ACID Properties in DBMS



CAP theorem

- **CAP theorem**: The following three attributes cannot be concurrently guaranteed by a distributed system (also known as Brewer's theorem)

  - Consistency: every node in the system sees the exact same data at the exact same moment.
  - Availability: Requests may always be answered by the system.
  - Partition Tolerance: the system keeps working even when there are many node communication failures.

➔ Any distributed system has to select one of these three attributes.
  - CA
  - CP
  - PA

See Annexes for more details

Limitations of HDFS

- **Requirements of a Database**

  - Structured: Rows and columns
  - Random access: Update one row at a time
  - Low latency: Very fast read/write/update operations
  - ACID compliant: Ensure data integrity

Hadoop is a big data processing framework

Hadoop is not a database!

Limitations of HDFS

Unfortunately, Hadoop makes a very poor database

- Basic structure exists for some file types (CSV, JSON, XML) Hadoop enforces **no constraints** on these

- **Cannot** create, access and modify individual records in a file
- **No random access** MapReduce parses entire files to extract information

- **Not suited for real-time** processing where a user waits for data to be retrieved
- **High latency**: Batch processing with long running jobs

HDFS is a file storage system and provides **no guarantees** for data integrity

Limitations of HDFS

- Hadoop has a bunch of limitations which does not allow it to be used stand alone in its bare-bone form as a storage and source of truth for data.
  - All data stored in Hadoop is unstructured.
  - Data can be stored in thousands, maybe millions of files. Hadoop has no idea of what exactly is within those files.
  - Hadoop does not allow to access a single record or even a series of records at random. The overhead or the time involved in seeking a particular record or a field is very high.
  - Hadoop offers no ACID compliance.

Limitations of HDFS

- Data in **HDFS has no schema**, no rows and columns, no tables. They're simply text files, log files, audio or video files. HDFS doesn't really care or know what's within the files that have stored in there. This flexibility is extremely powerful because then Hadoop can be used for any kind of storage in any kind of system
- But a major weakness when it comes to using Hadoop as a database is there might be files that have some kind of basic structure. **CSV's, XML or JSON files**. The developer who has to impose the structure upon the data. Hadoop has no features to help you with checking whether the files are well-formatted, well-structured, whether all the required needed fields are actually present. It's just free-form data. The structure and even the enforcement of the structure is up to the developer.
- Hadoop is suitable for **batch processing** where there are typically very long-running jobs working on millions of records at a time.
- The storage system **HDFS does one thing well. Stores huge amounts of data and backs them up reliably**. It offers no constraints or no guarantees for the integrity of data stored.

Hbase: presentation

- **HBase is a distributed database** because the data are stored in the form of files in HDFS.
- HBase extracts these files and presents **data in the form of records** which is indexed by something called a row key.
- **HBase is scalable** because the amount of disk space available for data storage is basically the sum total of the disc space of the individual machines in a cluster.
- **Hbase is reliable** because the fault tolerance and recovery features of Hbase simply piggybacks on those provided by Hadoop.
- Like a traditional database, Hbase is aware of the structure of data that's stored within it. The difference is that the data we **structure is very loosely defined**. There are no strict specifications on how many columns, and what kind of data needs to be stored on those columns, etc.
- Hbase has a way to index every record that's stored within it by use of something called the row keys. **Row keys allow real-time access** to every individual record or to a series of individual records within a certain reach.

Hbase: presentation

- **Hbase does not follow ACID** compliance as **strictly** as a traditional, relational database; however, there are certain transactions which can be said to have ACID properties.

- **HBase allows batch processing using MapReduce**. Hbase gives the **best of both worlds.**
  - Long running, very intensive processing jobs which will use MapReduce.
  - Real-time processing using the row keys with service indices for individual records.

- **Hbase applies to the following scenarios**

  - Massive data
  - The ACID feature are not required (contrary to RDBMs data bases)
  - High throughput
  - Efficient random reading of massive data
  - High scalability
  - Simultaneous processing structured and unstructured data

History

An	Event
2006	Google has published the document on BigTable.
2007	The initial HBase prototype was created as a Hadoop contribution.
2007	The first HBase usable with Hadoop 0.15.0 has been published.
2008	HBase has become a sub-project of Hadoop.
2008	HBase 0.18.1 has been released.
2009	HBase 0.19.0 has been released.
2009	HBase 0.20.0 has been released.
2010	HBase has become the Apache top-level project.

Hbase: presentation

How use HBase

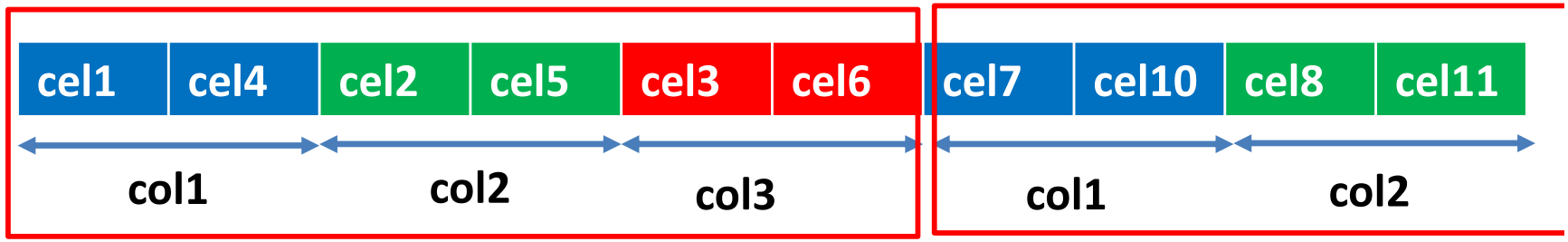
Hbase: presentation
Column-Oriented DBMS

HBase is a distributed, column-oriented DBMS that provides real-time read and write access to data stored on the HDFS.

	col1	col2	col3
Row1	cel1	cel2	Cel3
Row2	Cel4	cel5	cel6
Row3	cel7	cel8	Cel9
row4	cel10	cel11	cel12

Row oriented database

column oriented database



- Column-oriented databases offer a number of advantages over traditional relational databases. These include

- **Efficient storage**

  Column-oriented databases are optimized for efficient data storage. Unlike relational databases, where data is stored by row, column-oriented databases store data by column. This reduces the amount of storage space required, particularly when data is highly dispersed or some columns contain NULL values.

- **High performance for analytical queries**

  Column-oriented databases are particularly well suited to analytical queries requiring the analysis of large quantities of data. By storing data on a column-by-column basis, column-oriented databases can read only the columns required for a given query, resulting in high performance.

- **Horizontal scalability**

  Column-oriented databases are designed to be easily scalable horizontally. This means they can be distributed across multiple servers, allowing storage and processing capacity to be increased simply by adding new servers to the cluster.

- **Schema flexibility**

  Unlike relational databases, which require a fixed schema, column-oriented databases offer greater schema flexibility. This means that columns can be added or removed without having to modify the entire database structure.

- HBase is a **byte-in and byte-out database** that uses **the Put and Result interfaces to convert data into byte arrays.**
- It stores and gives out byte arrays, and the Put and Result methods handle encoding and decoding of objects.
- HBase can **store anything that can be converted into bytes,** from a simple string to an image file. It can store values up to **10 to 15 MB in an HBase cell.**
- It is not advisable to convert a huge file or value into byte arrays and store it in HBase. However, HDFS can be used to host files with underlying distribution and file metadata into an HBase table.
- HBase provides APIs for serializing and deserializing data to be put into an HBase table and fetched from an HBase table.

Hbase table

Row Key | Column Family

Row Key	Customers		Products		
Customer ID	Customer Name	City & Country	Product Name	Price	Column Qualifiers
1	Sam Smith	California, US	MIke	$500	Cell
2	Arijit Singh	Goa, India	Speakers	$1000	
3	Ellie Goulding	London, UK	Headphones	$800	
4	Wiz Khalifa	North Dakota, US	Guitar	$2500	

Tables: Data is stored in a table format in HBase. But here tables are in column-oriented format.

Row Key: Row keys are used to search records which make searches fast.

Column Families: Various columns are combined in a column family. These column families are stored together which makes the searching process faster because data belonging to same column family can be accessed together in a single seek.

Column Qualifiers: Each column's name is known as its column qualifier.

Cell: Data is stored in cells. The data is dumped into cells which are specifically identified by rowkey and column qualifiers.

Timestamp: Timestamp is a combination of date and time. Whenever data is stored, it is stored with its timestamp. This makes easy to search for a particular version of data.

Hbase data model

Hbase table

- **Table**
  - Contains column-families
- **Column family**
  - Logical and physical grouping of columns
- **Column**
  - Exists only when inserted
  - Can have multiple versions
  - Each row can have different set of columns
  - Each column identified by it's key
- **Row key**
  - Implicit primary key
  - Used for storing ordered rows
  - Efficient queries using row key

HBTABLE

Row key	Value
11111	cf_data: {'cq_name' : 'name1' , 'cq_val' : 1111} cf_info: {'cq_desc' : 'desc11111'}
22222	cf_data: {'cq_name' : 'name2' , 'cq_val' : 2013 @ ts = 2013, 'cq_val' : 2012 @ ts = 2012 }

HFile

11111	cf_data	cq_name	name1 @ ts1
11111	cf_data	cq_val	1111 @ ts1
22222	cf_data	cq_name	name2 @ ts1
22222	cf_data	cq_val	2013 @ ts1
22222	cf_data	cq_val	2012 @ ts2

HFile

11111	cf_info	cq_desc	desc11111 @ ts1

Example : For example cf_data and cf_info are two families of columns

- Defining a family of columns is very important. Thanks to the family of columns, we can define the structure of the files where the data will be stored.
- HBase will store the data on HDFS and further it will take advantage of all the power of HDFS in terms of data distribution, load distribution, fault tolerances etc.
- HBase, it will take care of data semi-structuring. So for each column family, we will create a different Hfile file. Each Hfile file represents a different family. (For example **cf_data** and **cf_info** are two families of columns. They will be stored in two different Hfile files.)
- If the size of Hfile exceeds the size limit of an HDFS block, the Hfile file will be subdivided into blocks. One can have not only several Hfile for a given but also several blocks for each Hfile file.

Hbase data model

Hbase table

Data in a Traditional Database

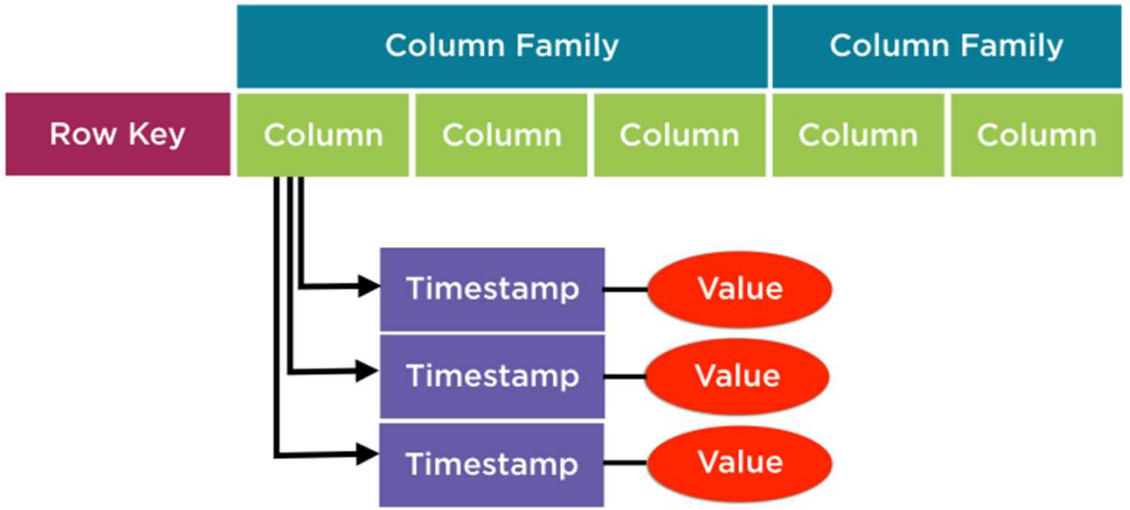
This is a **2-dimensional** data model

Id	To	Type	Content
1	mike	offer	Offer on mobiles
2	john	sale	Redmi sale
3	jill	order	Order delivered
4	megan	sale	Clothes sale

unique row id column name

HBase has a **4-dimensional** data model

Example



Hbase data model

Hbase table

Row Key

Columns

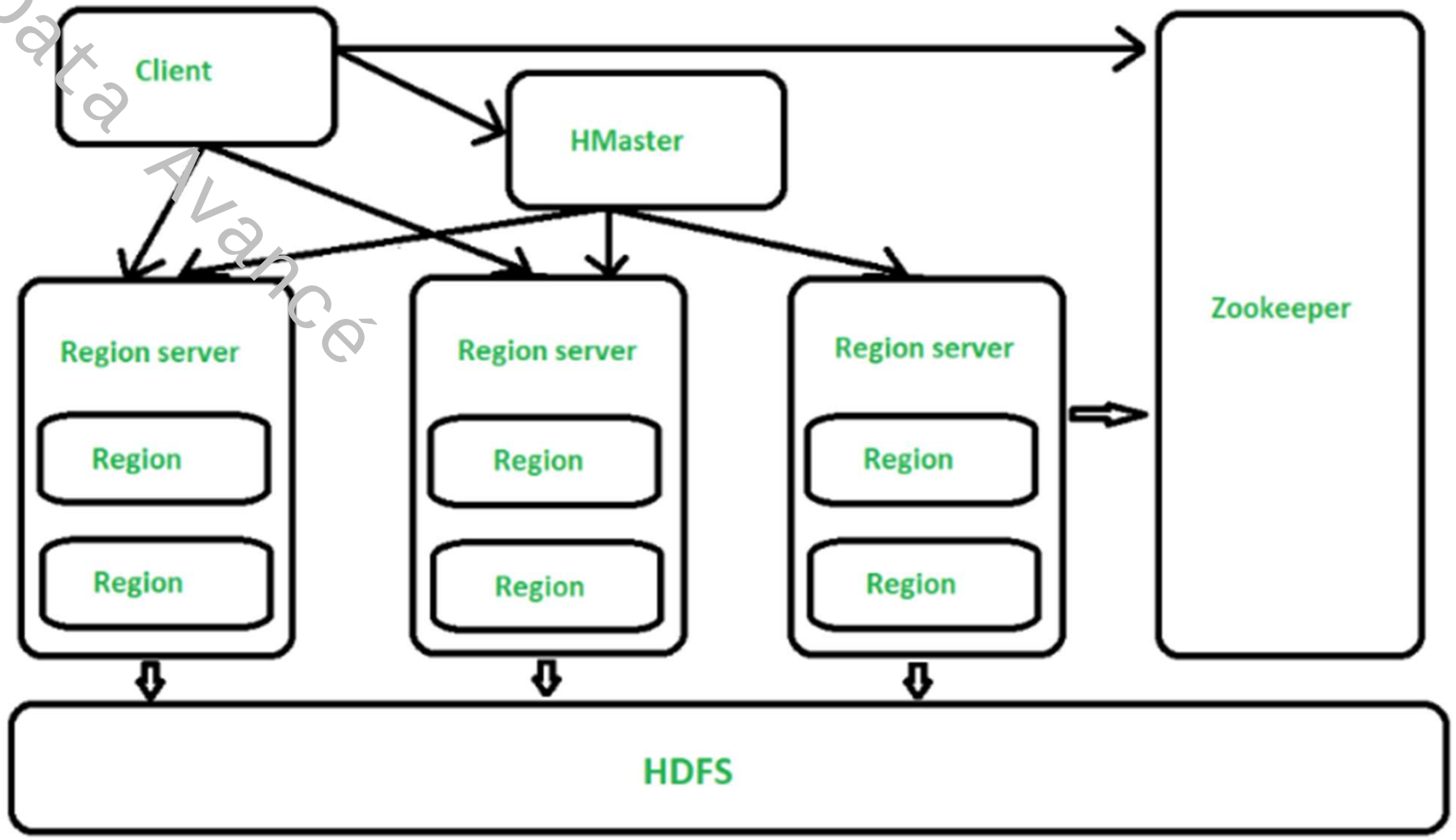
Id	To	Type	Content
1	mike	offer	Offer on mobiles
2	john	sale	Redmi sale
3	jill	order	Order delivered
4	megan	sale	Clothes sale

Value

Column family

Id	Column	Value
1	To	mike
1	Type	offer
1	Content	Offer on mobiles
2	To	john
2	Type	sale
2	Content	Redmi sale
3	To	jill
3	Type	order
3	Content	Order delivered
4	To	megan
4	Type	sale
4	Content	Clothes sale

Hbase Architecture



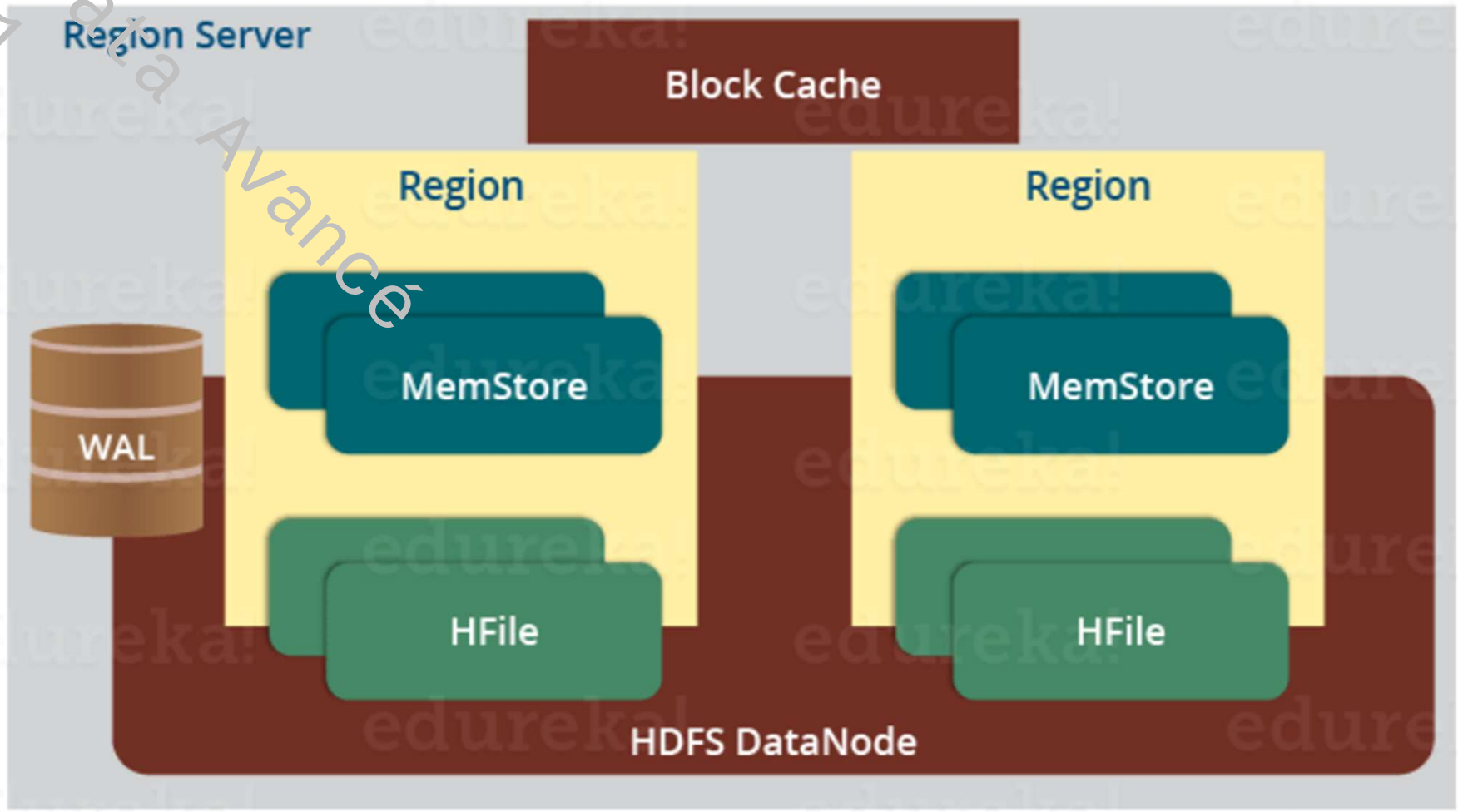
- **The RegionServer** is a crucial component of HBase architecture, responsible for hosting and serving a set of regions, which are the basic units of data storage in HBase. Each region is a part of a table and contains a range of rows. The RegionServer handles read and write requests for the data stored in its regions and is responsible for ensuring data consistency and availability. Here are some key aspects of the RegionServer:

  - Data Storage: Regions are the basic units of data storage in HBase. Each region is a part of a table and contains a range of rows. The RegionServer manages the storage and retrieval of data in these regions
  - Data Consistency and Availability: The RegionServer ensures data consistency and availability by handling read and write requests for the data stored in its regions. It is responsible for maintaining the integrity of the data and making it available to client processes
  - Region Splitting and Merging: When a table grows, HBase automatically splits the regions to distribute the write and query load across region servers. The RegionServer process is responsible for managing these splits and merges to maintain optimal performance and memory usage

- RegionServer Configuration: The RegionServer can be configured through various properties in the HBase configuration file. For example, the hbase.regionserver.handler.count property defines the number of threads the region server keeps open to serve requests to tables

- **Region :**
  - A region contains all the rows between the start key and the end key assigned to that region. HBase tables can be divided into a number of regions in such a way that all the columns of a column family is stored in one region. Each region contains the rows in a sorted order.
  - Many regions are assigned to a **Region Server**, which is responsible for handling, managing, executing reads and writes operations on that set of regions.

- Region server components



- **WAL (Write Ahead Log):**
  - A file attached to each Region Server.
  - Stores new data temporarily before it's permanently committed to disk.
  - Serves as a recovery mechanism in case of failures, ensuring data durability.
- **Block Cache:**
  - Resides in memory on the Region Server.
  - Caches frequently read data for faster access.
- **MemStore:**
  - A write cache for incoming data.
  - Each column family within a region has its own MemStore.
  - Sorts data in order before committing it to disk.
  - Once MemStore reaches a certain size, it flushes data to HFiles for permanent storage.
- **HFile:**
  - The actual on-disk storage format for data in HBase.
  - Resides on HDFS for distributed persistence.
  - Contains the cells (key-value pairs) for a region.

- Write Mechanism in HBase Architecture
  - The client will first have to find the Region Server and then the data's location for altering it. (This step is involved only for converting data and not for writing fresh information)
  - The actual write request begins at the WAL, where the client writes the data.
  - WAL transfers the data to MemStore and sends an acknowledgment to the user.
  - When MemStore is filled with data, it commits the data to HFile, where it is stored.

- Read Operations:
  - Client sends a read request to the Region Server.
  - The first scan is made at the read cache, which is the Block cache.
  - The next scan location is MemStore, which is the write cache.
  - If the data is not found in block cache or MemStore, the scanner will retrieve the data from HFile.

➔

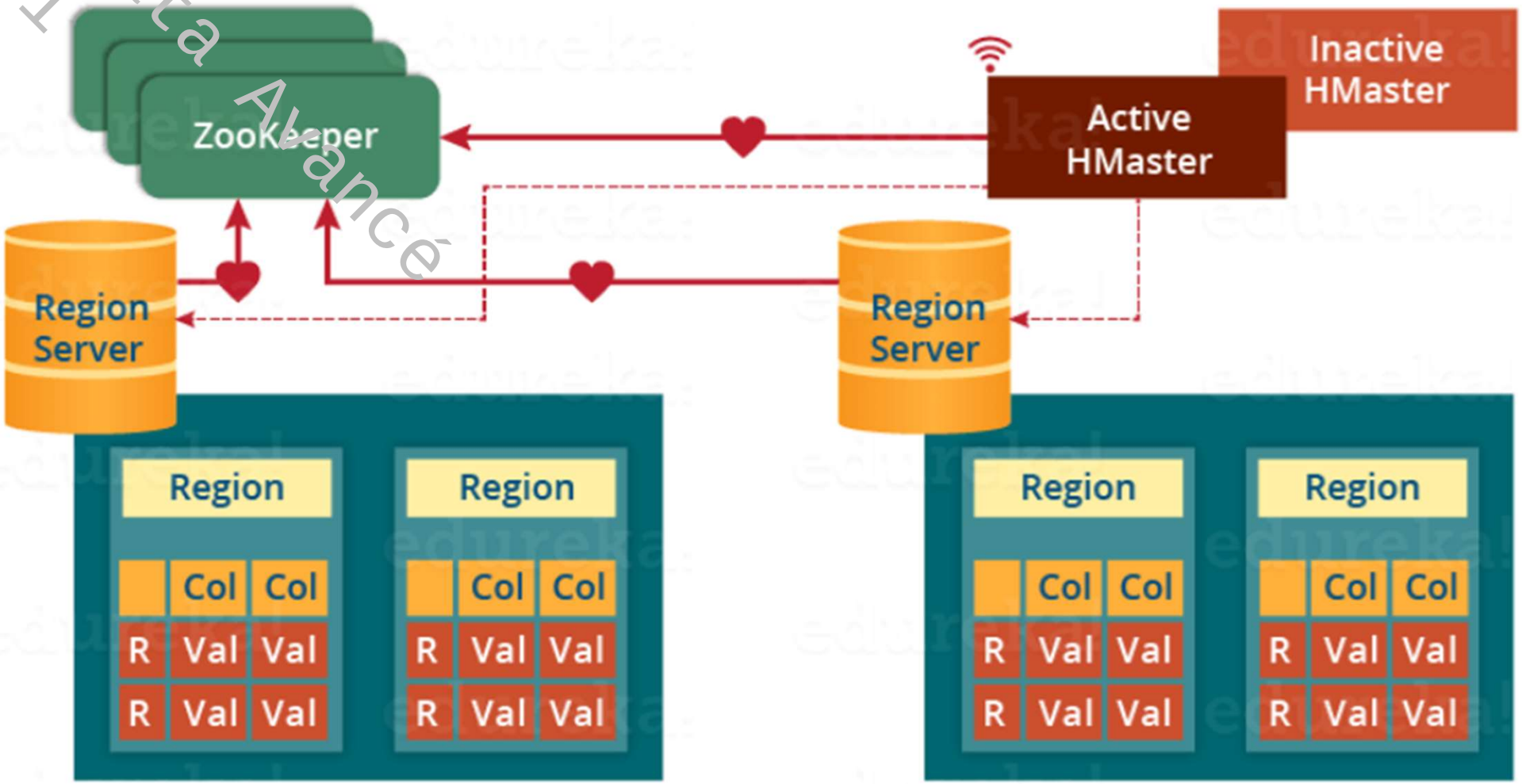
- Region Servers manage and serve data in regions, distributed across HDFS.
- WAL ensures data durability in case of failures.
- Block Cache speeds up read operations by caching frequently accessed data.
- MemStore buffers writes before flushing them to HFiles for permanent storage.
- HFiles store data persistently on HDFS.

Hbase Architecture

- **HMaster**
  - HMaster handles a collection of Region Server which resides on DataNode.
  - HMaster performs DDL operations (create and delete tables) and assigns regions to the Region servers as you can see in the above image.
    - It coordinates and manages the Region Server (similar as NameNode manages DataNode in HDFS).
    - It assigns regions to the Region Servers on startup and re-assigns regions to Region Servers during recovery and load balancing.
    - It monitors all the Region Server's instances in the cluster (with the help of Zookeeper) and performs recovery activities whenever any Region Server is down.
    - It provides an interface for creating, deleting and updating tables.

  - HBase has a distributed and huge environment → HMaster alone is not sufficient to manage everything → ZooKeeper

- ZooKeeper



•Zookeeper acts like a coordinator inside HBase distributed environment. It helps in maintaining server state inside the cluster by communicating through sessions.

•Every Region Server along with HMaster Server sends continuous heartbeat at regular interval to Zookeeper and it checks which server is alive and available. It also provides server failure notifications so that, recovery measures can be executed.

• Inactive Hmaster acts as a backup for active server. If the active server fails, it comes for the rescue.

•The active HMaster sends heartbeats to the Zookeeper while the inactive HMaster listens for the notification send by active HMaster. If the active HMaster fails to send a heartbeat the session is deleted and the inactive HMaster becomes active.

•if a Region Server fails to send a heartbeat, the session is expired and all listeners are notified about it. Then HMaster performs recovery actions

•Zookeeper also maintains the .META Server's path, which helps any client in searching for any region. The Client first has to check with .META Server in which Region Server a region belongs, and it gets the path of that Region Server.

- Examples of how Zookeeper is used by HMaster:
  - When HBase starts, HMaster authenticates to Zookeeper for a list of existing RegionServers. HMaster then uses this list to distribute regions across RegionServers.
  - If a RegionServer fails, Zookeeper notifies HMaster. HMaster can then use Zookeeper to reassign regions hosted by the failing RegionServer to other RegionServers.
  - If a RegionServer is decommissioned for maintenance, HMaster can use Zookeeper to mark the RegionServer as unavailable. This prevents clients from submitting requests to the RegionServer.

➔ Zookeeper is a key component of the HBase architecture. It enables HBase to manage cluster status reliably and efficiently, which is critical for system scalability, performance and reliability.

Hbase : interaction

- Interaction with HBase

  - **HBase Shell**:
    The HBase Shell is a command-line interface that allows you to interact with an instance of HBase. It is installed with HBase and is accessible f rom the command line. This tool is useful for managing HBase tables and data interactively, performing tasks such as creating and modifying table properties, and adding, modifying or deleting data.

  - **Java API**:
    HBase offers a Java API that allows developers to interact with HBase using languages such as Java, Python, Ruby, C++, etc. This API provides CRUD operations for HBase tables and data.

  - **Apache Phoenix**:
    Apache Phoenix is an open-source extension to Apache HBase that provides a SQL query layer. It allows to interact with HBase tables using standard SQL queries, without having to write HBase-specific code.

Hbase : Shell commands

- **Table Management Commands** :
  - create '<table_name>', '<column_family_1>', '<column_family_2>'
    create 'students', 'info', 'grades'
  - List Tables :
    list
  - Deactivating a Table:
    disable '<table_name>'
    disable 'students'
  - Activation Table :
    enable '<table_name>'
    enable 'students'
  - Drop Table :
    drop '<table_name>'
    drop 'students'

Hbase : Shell commands

- **Data Management Orders:**
  - Insertion of Data:

put '<table_name>', '<row_key>', '<column_family:column_qualifier>', '<value>'
put 'students', 'john_doe', 'info:name', 'John Doe'

  - Obtaining Data:

    get '<table_name>', '<row_key>'
    get 'students', 'john_doe'

  - Scans Table :

    scan '<table_name>'
    scan 'students'

  - Data Suppression :

delete '<table_name>', '<row_key>', '<column_family:column_qualifier>'
delete 'students', 'john_doe', 'info:name'

Hbase : Shell commands

- **Column Management Commands and Column Families:**

  - List of Column Families:
    list '<table_name>'
    list 'students'
  - Addition of Column Family:
    alter '<table_name>', NAME => '<column_family_name>'
    alter 'students', NAME => 'contact'
  - Deleting Column Family:

alter '<table_name>', {NAME => '<column_family_name>', METHOD => 'delete'}
alter 'students', {NAME => 'contact', METHOD => 'delete'}

Hbase : Shell commands

- **HBase Feature Management commands:**
  - Status Cluster :
    status
  - Command help :
    help '<command_name>'
    help 'put'
  - Version Management Commands:
    get '<table_name>', '<row_key>', {VERSIONS => <num_versions>}
    get 'students', 'john_doe', {VERSIONS => 3}

https://learnhbase.wordpress.com/2013/03/02/hbase-shell-commands/

Annex : ACID theorem examples

- **Atomicity** is the most important property of the ACID theorem. It guarantees that transactions are executed indivisibly. This means that if part of a transaction fails, the entire transaction fails and the database remains unchanged.

- For example, if the purpose of a transaction is to transfer money from one account to another, atomicity guarantees that the money is either transferred from one of the accounts or from the other, but never from one account to the other without affecting the other account.

Before: X : 500	Y: 200
Transaction T	
T1	**T2**
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

The money has been taken out of X but not added to Y if the transaction fails after T1 but before T2 (that is, after write(X) but before write(Y)). The outcome is an inconsistent state of the database. Therefore, to guarantee the correctness of the database state, the transaction must be conducted in its entirety.

Annex : ACID theorem examples

- **Consistency** ensures that the database is always in a consistent state after a transaction has been executed. This means that the data in the database respects the consistency constraints defined by the database administrator.

- For example, a consistency constraint might stipulate that an account balance can never be negative. Atomicity and consistency ensure that if a transaction attempts to withdraw more money from an account than its balance, the transaction will fail and the account balance will not be affected.

- For example (Referring to the last example)
  - The total amount before and after the transaction must be maintained.
  - Total before T1 occurs = 500 + 200 = 700.
  - Total after T2 occurs = 400 + 300 = 700.
  - Therefore, the database is consistent. Inconsistency occurs in case T1 completes but T2 fails. As a result, T is incomplete.

Annex : ACID theorem examples

- **Isolation** ensures that transactions run in isolation from each other. This means that changes made by a transaction are only visible to that transaction until it is committed.
- Isolation is important to avoid conflicts between transactions. For example, if two transactions attempt to modify the same line of data, isolation ensures that each transaction's modifications will be visible to the other transactions once they have been committed.

T	T''
Read (X)	Read (X)
X: = X*100	Read (Y)
Write (X)	Z: = X + Y
Read (Y)	Write (Z)
Y: = Y – 50	
Write (Y)	

- Given X = 500 and Y = 500. Think about the two transactions, T and T''.
- Assume that once T has been run through Read (Y), T'' begins → Consequently, interleaving of operations occurs, causing T'' to read the proper value of X but the wrong value of Y.
- As a result, the sum calculated by T'' (X+Y = 5000 + 500 = 5500) is thus not consistent with the sum at end of the transaction that should be 5000 + 450 = 5450

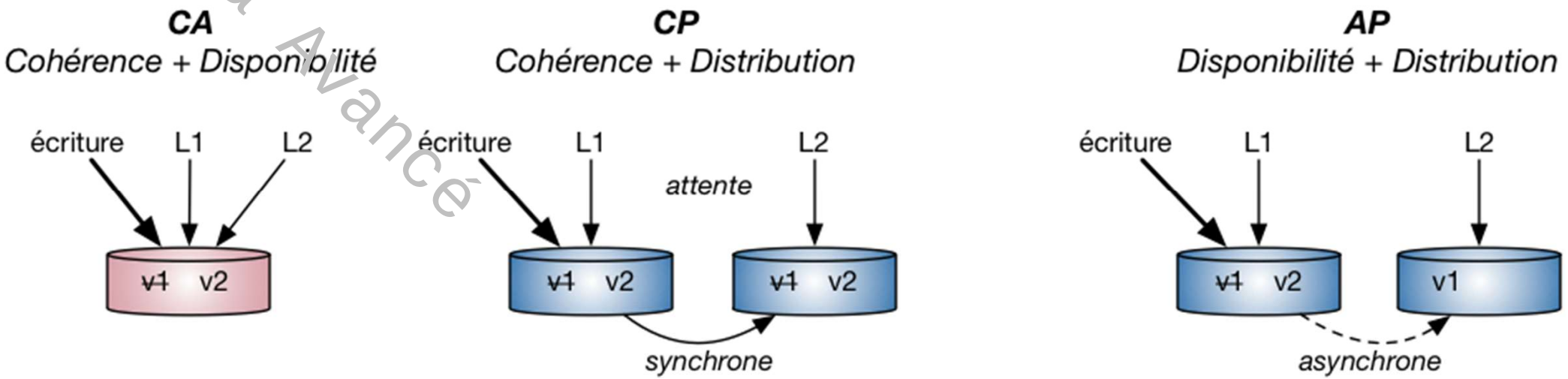
Annex : ACID theorem examples

- **Isolation** Durability ensures that the changes made by a transaction are persistent, even in the event of a system failure. This means that if the system crashes during the execution of a transaction, the changes made by the transaction will be restored once the system has restarted.
- Persistence is important to ensure data consistency even in the event of a system failure.

- Example: Stock management: stock management is a complex operation that requires the guarantee of atomicity, consistency, isolation and durability.
  - Atomicity ensures that changes to stock are either applied or reversed.
  - Consistency ensures that the stock is always correct.
  - Isolation ensures that changes made to inventory by different transactions are consistently visible.
  - Durability ensures that changes to inventory are persistent even in the event of a system failure.

Annex : CAP theorem

- In any database, you can only respect a maximum of 2 properties out of consistency, availability and distribution.

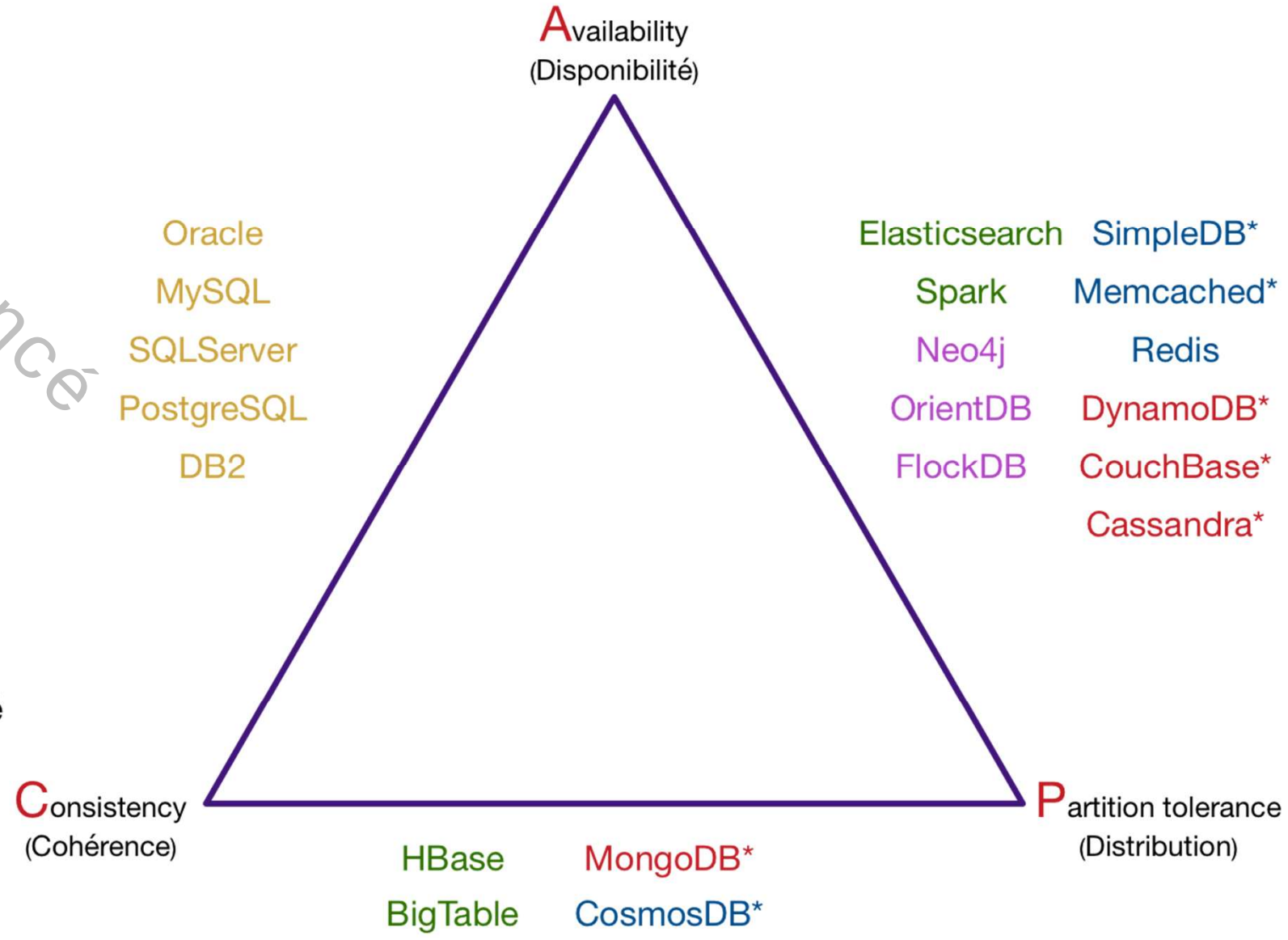


- **CA (Consistency-Availability)**
  - CA pair represents the fact that during concurrent operations on the same data, L1 and L2 queries return the new version (v2) without waiting.
  - This combination is only possible in the context of transactional databases such as **RDBM**.

Annex : CAP theorem

- **CP (Consistency-Partition Tolerance)**
  - CP couple now offers the possibility of distributing data over several servers while guaranteeing fault tolerance (replication).
  - At the same time, it is necessary to check the consistency of the data by guaranteeing the value returned despite competing updates.
  - Managing this consistency requires a protocol for synchronizing replicas, which introduces latency into response times (L1 and L2 wait for synchronization before seeing v2).
  - This is the case with the **MongoDB, CosmosDB**, **Hbase**, **BigTable**
- AP (Availability-Partition Tolerance)
  - AP aims to provide a fast response time while distributing data and replicas.
  - Updates are asynchronous over the network, and the data is 'Eventually Consistent' (L1 sees version v2, while L2 sees version v1).
  - This is the case with **Cassandra**, whose response times are appreciable, but the result is not 100% guaranteed when the number of simultaneous updates becomes large.
  - Case of: **Spark**, **Neo4js, DynamoDB, Redis, Elasticksearch, …**

Annex : CAP theorem



Modèle

Relationnel
Orienté Clé-Valeur
Orienté Colonne
Orienté Document
Orienté Graphe

* Possibilité de changer le mode de cohérence
Cohérence <-> Disponibilité

Annex : Examples related to HBase architecture

1. Example of horizontal scaling: Let's say a company initially sets up an Hbase cluster with three machines. As their data grows, they decide to scale horizontally by adding two more machines to the cluster. With this additional hardware, Hbase can distribute the workload across all five machines, improving performance and capacity.
2. Example of RegionServer functionality: Suppose a social media platform is using Hbase to store user posts and comments. Each user's data is divided into regions, with each region containing a subset of posts. When a user requests their posts, the RegionServer responsible for that region retrieves the data from its memory or disk storage and serves it to the user.
3. Example of HMaster coordinating failover: Imagine a Hbase cluster with three RegionServers, each hosting multiple regions. If one of the RegionServers goes down due to a hardware failure, the HMaster detects the failure and reassigns the affected regions to the other two RegionServers. This ensures that the data remains available and accessible even in the presence of failures.

4. Example of metadata management: Let's say a company has multiple tables in their Hbase system, each containing different types of data. The HMaster keeps track of the metadata for these tables, including the number and location of regions for each table. This metadata allows Hbase to efficiently handle read and write operations by knowing where each piece of data is stored in the cluster.

5. Example of fault-tolerance: Suppose a financial institution uses Hbase to store transaction data. By replicating data across multiple RegionServers in the cluster, Hbase provides fault-tolerance. If one machine or RegionServer fails, the replicated data on other machines can still be accessed, ensuring that no transaction data is lost and maintaining system reliability.

6. Example of HDFS integration: Imagine a healthcare organization analyzing a large dataset of patient records using Hbase. The combination of HDFS and Hbase allows them to store and process this massive amount of data efficiently. HDFS handles distributed storage, while Hbase provides fast access and processing capabilities, enabling the healthcare organization to perform complex analytics and gain valuable insights from the data.